# PGE Architecture

This document explains how Pinto's Game Engine is built and why it was built that way. Read it if you want to change the engine itself. If you only want to make a game with it, read the Guide instead.


1. Goals
2. The big picture
3. Packages
4. Starting up and the game loop
5. Threads
6. Graphics
7. Input
8. Tile maps and the camera
9. Scenes and game objects
10. Buttons and other screen elements
11. Sound
12. Files, tools and the build
13. Error messages
14. Design decisions
15. Where to add new features
16. What the engine cannot do yet

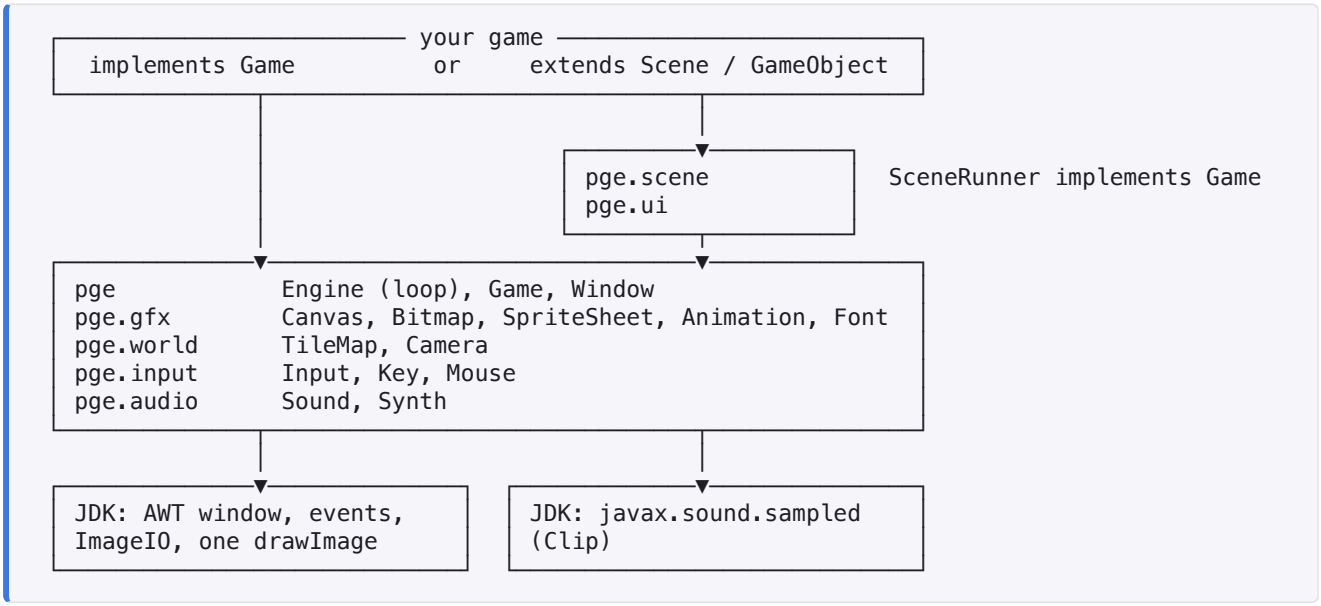

## 1. Goals

PGE was written for a student in a first-year programming course. The student should be able to make a game with it straight away, and later be able to read and change any part of it. Every other decision follows from that.

These are the rules the code follows:

- It uses only the parts of Java taught in a first course: classes, interfaces, simple inheritance, arrays, `ArrayList`, loops and `if` statements. It does not use lambdas, streams or other advanced features. Fields are private and have getters and setters.
- The engine draws every pixel itself, by writing numbers into an array. Java's own libraries are only used to open the window, read the keyboard and mouse, play sound and put the finished picture on the screen.
- It needs nothing except Java 25. There are no extra libraries to install, and no build tool such as Maven or Gradle.
- The game programmer writes methods, and the engine calls them at the right time. There are two ways to make a game: implement the `Game` interface, or build it out of `Scene`s and `GameObject`s. The second way is built using only the first.
- When something goes wrong, the error message says what happened in plain English and, where possible, how to fix it. A missing sound card never stops a game.

## 2. The big picture




Read the diagram from the top down. Your game uses the engine, and the engine uses Java's own libraries (the JDK). Games made with scenes go through the `pge.scene` package first.

The engine has 24 classes and about 4,000 lines, many of them comments. Only four files use Java's window, input or sound classes directly. Everything else is ordinary Java code.

| File | Java classes it uses | What for |
|---|---|---|
| `Window.java` | `Frame`, `Canvas`, `BufferStrategy`, `BufferedImage`, `GraphicsDevice` | opening the window, fullscreen, showing each picture |
| `input/Input.java` | `KeyListener`, `MouseListener`, `MouseMotionListener`, `MouseWheelListener`, `FocusListener` | finding out about key presses and mouse movement |
| `gfx/Bitmap.java` | `ImageIO`, `BufferedImage` | reading PNG files |
| `audio/Sound.java` | `AudioSystem`, `Clip`, `FloatControl` | reading WAV files and playing sound |

(`Key` and `Mouse` also mention Java classes, but only to copy the numbers Java uses for each key and mouse button.)

## 3. Packages

A package is a folder of related classes. PGE has seven:

| Package | Classes | What it does | Uses these other packages |
|---|---|---|---|
| `pge` | `Engine`, `Game`, `Window` | runs the game loop, starts and stops the game, owns the window | gfx, input, scene |
| `pge.gfx` | `Canvas`, `Color`, `Bitmap`, `SpriteSheet`, `Animation`, `Font` | pixels, pictures and text | none |
| `pge.input` | `Input`, `Key`, `Mouse` | keyboard, mouse and touch | none |
| `pge.world` | `TileMap`, `Camera` | levels, and which part of them is on screen | gfx |
| `pge.scene` | `Scene`, `GameObject`, `SceneRunner` | the second way to build a game, out of objects | pge, gfx, world, ui |
| `pge.ui` | `UIElement`, `Box`, `Label`, `Button`, `Picture` | buttons, text and panels on the screen | gfx, input |
| `pge.audio` | `Sound`, `Synth` | sound effects and music | none |

`pge.gfx`, `pge.input` and `pge.audio` do not use any other part of the engine, so you can read and understand each of them on its own.

`pge` and `pge.scene` use each other. `Engine.run(scene, ...)` creates a `SceneRunner`, and `SceneRunner` asks `Engine` for the screen size. This is the only place where two packages depend on each other, and it was done on purpose to keep starting a game with a scene simple.

## 4. Starting up and the game loop

### Starting up

When a game calls `Engine.run(game, width, height, title)`, the engine:

1. Creates the `Window`. The window makes a `BufferedImage` (Java's class for a picture in memory) that is `width` by `height` pixels, and takes the `int[]` array inside it that holds the colours.
2. Gives that same array to a new `Canvas`. From now on, anything the game draws on the canvas goes straight into the picture that will be shown. Nothing needs to be copied.
3. Calls `game.init()`.
4. Runs the game loop (below) until the window is closed or the game calls `Engine.stop()`.
5. Closes the window and ends the program.

`Engine.run(scene, ...)` does the same thing, but first wraps the scene in a `SceneRunner` (see section 9).

### The game loop

The game loop is the part of the engine that keeps the game going. In simplified form it looks like this:

```
nextUpdate = now
while the game is running:
    if it is not yet time for nextUpdate: wait a little, then check again
    while it is time for nextUpdate (at most 5 times in a row):
        Input.update()
        check the engine's own keys (F11, Alt+Enter, F3)
        game.update()
        nextUpdate = nextUpdate + 1/60 of a second
    if we are still behind: nextUpdate = now + 1/60 of a second
    game.draw(canvas), then show the picture in the window
```

The important ideas:

- Updates happen exactly 60 times per second, however fast or slow the computer is. Game code never has to measure time, so `x = x + 2` always means "2 pixels per update". This makes game code simpler, and the game behaves the same on every computer.
- If drawing a frame took too long, the loop runs several updates in a row before drawing again, so the game does not slow down. It never runs more than 5 in a row. After a long pause (for example, if the laptop went to sleep) it simply carries on from "now" instead of rushing to catch up.
- `Thread.sleep` is not very exact: it can wait a millisecond or more too long. So the loop sleeps for most of the waiting time and then checks the clock repeatedly for the last millisecond.
- The game is drawn once after each group of updates. Drawing more often would only show the same picture again.

### Stopping

Pressing the window's close button sets a flag that the loop checks. Calling `Engine.stop()` makes the loop end too. If the game's code crashes with any error, the engine closes the window, prints the error and ends the program. Without this, the window would stay on the screen, frozen.

## 5. Threads

A *thread* is a sequence of instructions that runs at the same time as other threads. Every Java program starts with one thread, the main thread. PGE runs the whole game on the main thread, so game code never needs to think about threads.

But Java itself starts some other threads:

| Thread | Started by | Does it touch the engine's data? |
|---|---|---|
| the event thread | Java's window system (AWT) | yes: it reports key presses and mouse movements to `Input`, and tells `Window` when the close button is pressed |
| sound threads | Java's sound system, one for each sound that is playing | no: each sound gets its own copy of the data it plays |
| other helper threads | Java | no |

When two threads use the same variables at the same time, things can go wrong in ways that are very hard to find. PGE avoids this in two places:

- `Input` keeps two sets of variables. The event thread only writes to the "pending" set. Once per update, the main thread copies "pending" into "current". The methods that use the pending set are marked `synchronized`, which means Java lets only one thread run them at a time. Game code only ever reads "current", which only the main thread changes, so the values stay the same for the whole update.
- The flag that says "the close button was pressed" is marked `volatile`. This makes sure that when the event thread changes it, the main thread sees the change straight away.

## 6. Graphics

### The picture is an array

The picture on the screen is one `int[]` array with one colour for every pixel, stored row by row. For a picture `width` pixels wide, the pixel at `(x, y)` is at index `y * width + x`. The same array is inside a `BufferedImage`, which `Window` creates like this:

```
image  = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);
pixels = ((DataBufferInt) image.getRaster().getDataBuffer()).getData();
```

Once per frame, `Window.present()` shows this image in the window with a single call to `drawImage`, made bigger to fill the window. It draws into a hidden copy of the window first and then swaps it in (this is what `BufferStrategy` does), so you never see a picture that is only half drawn.

### Making the picture bigger

The window is usually much bigger than the game picture. The engine makes the picture bigger by the largest whole number that still fits (2, 3, 4...), so every game pixel becomes a perfect square of screen pixels. Any space left over becomes black bars at the sides. Only when the window is smaller than the game does it shrink the picture by a fraction. The scaling is set to "nearest neighbour", which copies pixels instead of blending them, so the picture stays sharp.

### How fast is it?

At 640 x 360 the picture has 230,400 pixels. The example games take about 2.3 milliseconds per frame to draw and show it. At 60 frames per second there are 16.7 milliseconds available, so there is plenty of time left. Because our own code writes every pixel, a bigger picture takes longer. About 960 x 540 is the largest size that still runs comfortably without clever tricks.

## Colours and see-through pixels

A colour is an `int` written as `0xRRGGBB`: two hexadecimal digits each for red, green and blue. `Color.TRANSPARENT` is `-1`. No real colour has that value, so the engine uses it to mean "draw nothing here".

When a `Bitmap` loads a PNG file, it decides once which pixels are see-through: pixels that are more than half transparent in the PNG, and pixels of the colour key if one was given. Those pixels are stored as `TRANSPARENT`. When drawing, the engine only has to check each pixel against `-1`. Pixels are either fully drawn or not drawn at all; the engine does not blend colours (yet).

## Canvas

All drawing goes through the `Canvas` class. Every drawing method does three things:

1. It subtracts the camera position, to turn world positions into screen positions.
2. It skips any part that is outside the picture, so games can safely draw things that are half off the screen. This is called *clipping*.
3. It writes the colours into the `pixels` array.

| Drawing | How it is done |
|---|---|
| rectangles | one row at a time, using `Arrays.fill` |
| lines | Bresenham's line algorithm, which only needs whole numbers |
| circles | the midpoint circle algorithm: work out one eighth of the circle and mirror it into the other seven |
| pictures | two loops, over the rows and the columns of the picture, skipping see-through pixels and anything off screen |
| silhouettes | like pictures, but every visible pixel is drawn in the same colour |
| text | each letter is a small picture; each visible pixel of it is drawn as a square |

## Pictures, sprite sheets, animation and fonts

- A `Bitmap` is a width, a height and an `int[]` of colours, just like the canvas.
- A `SpriteSheet` cuts one big picture into equal pieces as soon as it is created. After that, `get(i)` just returns piece number `i` from an array.
- An `Animation` counts how many times its `update()` has been called. Dividing that count by `updatesPerFrame` tells it which picture to show. Counting updates instead of measuring time keeps it in step with the game loop.
- A `Font` is a sprite sheet with one letter per piece: the 96 characters with codes 32 to 127, in a grid of 16 by 6. When it loads, it measures how wide each letter is, so narrow letters like `i` take up less space than wide ones like `m`. Character 127 is a box, shown for any letter the font does not have. The default font, `pge/gfx/font.png`, is loaded the first time text is drawn.

# 7. Input

Java gives every key a number (`Key` gives these numbers names like `Key.LEFT`). `Input` stores the state of every key in arrays of 1,024 `boolean`s, indexed by that number. Mouse buttons are stored the same way, in arrays of 8.

`Input` keeps the state from this update ("current") and from the update before ("previous"). From those two, it can answer every question:

| Question | Answer |
|---|---|
| `isDown` | is it down now? `current` |
| `wasPressed` | is it down now, but was not before? `current && !previous` |
| `wasReleased` | was it down before, but not now? `previous && !current` |

There is one more problem. A very quick tap can press and release a key between two updates. If `Input` only looked at whether the key is down right now, it would miss the tap completely. So there is an extra flag that remembers "this key was pressed since the last update", which makes the tap count as down for one update.

### Where is the mouse?

Java reports the mouse position in window pixels, but the game wants game pixels. Every frame, `Window.present()` tells `Input` where the game picture is inside the window and how big it is (`setScreenArea`). Then `Input.update()` converts the position:

```
gameX = (windowX − pictureLeft) ∗ gameWidth / pictureWidth
```

`isPointerInside()` is false when the mouse is over the black bars or outside the window. On a touch screen, Java reports a finger as if it were the left mouse button, so the "pointer" methods work for touch too. When the window stops being the active window, every key and button is set to "up", because Java will not report any key releases that happen while another window is active.

# 8. Tile maps and the camera

### TileMap

A `TileMap` stores a grid of tile numbers. Like the picture, the grid is kept in a normal one-dimensional array: the tile in column `c` and row `r` is at index `r ∗ columns + c`. The map also keeps the `SpriteSheet` with the tile pictures, and a `boolean[]` array that says which tile numbers are walls. The number `−1` (`EMPTY`) means there is no tile.

- Maps are loaded from CSV files, with one row of the grid on each line and `-1` for empty places. This is the format the Tiled editor exports. If a file has a mistake, the error message gives the file name, the line and the position.
- To find which column contains a pixel position `x`, the map uses `Math.floorDiv(x, tileWidth)`. Normal division would be wrong for negative numbers: `-1 / 16` is `0`, which would put a point just left of the map inside column 0, while `Math.floorDiv(-1, 16)` is `-1`, which is correctly outside the map.
- `overlapsSolid(x, y, width, height)` checks every tile that a rectangle touches. By default, anything outside the map counts as a wall, so nothing can leave it.
- When drawing, the map works out which columns and rows the camera can see and only draws those. So a huge map takes no longer to draw than a small one.

The engine does not move characters or stop them at walls. The example games do that themselves: they move a character one pixel at a time and ask `overlapsSolid` before every step (see `games/platformer/Main.java` and `games/platformer-scenes/Hero.java`). This was left out of the engine on purpose. It is the most interesting code in a platformer, and the student should write it and understand it.

### Camera

The camera stores the world position that is shown in the top-left corner of the screen. It keeps this as a `double`, and rounds it down when it gives it to the canvas.

Each time `follow(targetX, targetY)` is called, the camera does three things, once for `x` and once for `y`:

1. Dead zone. If the target is inside a box in the middle of the screen, the camera wants to stay where it is. If the target is outside the box, the camera wants to move just far enough to bring the target back to the edge of the box. With no dead zone, this means "put the target exactly in the middle".
2. Smoothing. The camera moves part of the way towards where it wants to be: `position = position + (wanted - position) * smoothness`. With a smoothness of 0.15, it covers 15% of the remaining distance every update. Big distances shrink quickly and small ones slowly, so the camera slows down gently as it arrives. A smoothness of 1.0 moves it there straight away.
3. Limits. The camera is kept between 0 and `worldSize - screenSize`, so it never shows anything outside the level. If the level is smaller than the screen, the level is shown in the middle instead.

`centerOn` moves the camera without smoothing (useful when a level starts). `toWorldX` and `toWorldY` turn mouse positions into world positions. `canSee` tells you whether a rectangle is on the screen.

## 9. Scenes and game objects

`Scene` and `GameObject` are optional. They are built only from things the engine already offers to every game. `SceneRunner` connects them to the engine: it implements the `Game` interface, and its `update()` and `draw()` simply call the current scene's. To the engine, a game made of scenes is just another `Game`.

## What a scene holds

- `layers`: tile maps, drawn first, in the order they were added
- `objects`: the game objects that are in play
- `joining`: objects that were added during this update and are waiting to join
- `uiElements`: buttons, labels and other screen elements, drawn last
- a `Camera`. It is created when the scene starts, not when it is constructed, because a scene can be constructed before the engine knows the screen size (for example in `main`).

## What happens in each update

1. The screen elements update, so that `button.wasClicked()` already has the right answer in step 3.
2. Every object's `update()` runs, and then its animation moves forward.
3. The scene's own `update()` runs.
4. Removed objects are taken out of the list. Then the objects in `joining` are added to the list, and their `start()` method runs. If a `start()` method adds more objects, they join in this same step.

Adding and removing objects is always put off until step 4. This means game code can call `add()` and `remove()` anywhere, even inside a loop over `getObjects()`. Changing a list while a `for` loop is going through it would normally crash with a `ConcurrentModificationException`, a common surprise for beginners.

In step 4, removed objects are taken out by looping through the list backwards. Going forwards would skip an object every time one is removed, because removing an item moves all the items after it one place forward.

## What happens in each frame

1. `drawBackground(canvas)` runs. By default it fills the screen with the background colour.
2. With the camera on: the layers are drawn, then the objects, in order of their depth, lowest first.
3. With the camera off: the screen elements are drawn, then `drawUI(canvas)` runs.

The objects are sorted by depth with an insertion sort: each object is moved left past every object with a higher depth. Objects with the same depth stay in the order they were added. Since the order rarely changes from one frame to the next, the list is almost always sorted already, and insertion sort is very fast in that case.

## Finding objects

Every object can have a name. `find(name)` returns the first object with that name, and `findAll(name)` returns all of them. To use the object as its own class, you cast it: `(Hero) getScene().find("hero")`. Java has a more advanced way that avoids the cast, but it needs generics, which a first course usually does not cover.

### Changing scenes

`Engine.setScene(next)` only remembers the request. At the start of the next update, `SceneRunner` makes the switch: it prepares the new scene (camera and screen size), calls its `start()`, and lets the objects added in `start()` join straight away, so they are all there for the first update.

## 10. Buttons and other screen elements

`UIElement` is an abstract class: it holds what every screen element has (a position, a size, and whether it is visible), but it cannot be used on its own. Every kind of element extends it and writes its own `draw(Canvas)` method. Elements are drawn in screen positions, in the order they were added to the scene.

A `Button` remembers three things, and updates them once per update:

| Variable | Meaning | When it becomes true |
|---|---|---|
| `hovered` | the mouse is over the button | checked again every update |
| `pressed` | the mouse button went down on this button and has not been let go | the mouse button is pressed while `hovered` |
| `clicked` | the button was clicked; true for one update only | the mouse button is let go while the button is both `pressed` and `hovered` |

If you press the mouse button on a button but let go somewhere else, it does not count as a click. Buttons in other programs work the same way. Games ask a button whether it was clicked (`wasClicked()`), the same way they ask `Input` whether a key was pressed.

A `Label` measures how big its text is every time the text, size or font changes. This is what makes centred labels and mouse checks on labels exact.

## 11. Sound

### Sound

A `Sound` keeps the sound data in memory, as 16-bit numbers. Java plays sounds with a `Clip`, which can only play one copy of a sound at a time. So each `Sound` opens 4 clips (we call them voices). `play()` uses a voice that is not busy. If all four are busy, it starts again with the one whose turn it is. This way the same sound can play up to four times on top of itself. `loop()` plays the first voice again and again.

- When loading a file, the sound is converted to 16-bit numbers, a format that every sound card accepts. Java can read WAV, AIFF and AU files.

- The volume of a clip is the sound's own volume times the master volume. Sound cards measure volume in decibels, so the engine converts it with the formula `20 * log10(volume)`. Every sound that is created is kept in a list, so that `setMasterVolume` can also change the volume of sounds that are already playing.
- If the clips cannot be opened, usually because the computer has no working sound output, the sound gets no voices at all. The engine prints one warning (only once for the whole game), and from then on every method of that sound does nothing.

### Synth

`Synth.tone(wave, startHz, endHz, seconds, volume)` makes an array of numbers between -1 and 1, 22,050 of them for every second of sound. A sound wave repeats the same shape over and over. The frequency, measured in hertz (Hz), is how many times per second it repeats.

- The code keeps a number called the *phase* that says how far through the current repeat it is, from 0 to 1. For every number it produces, the phase moves forward by `frequency / 22050`. When it passes 1, a new repeat begins. The wave shape turns the phase into a value: for example, a square wave is 1 for the first half of the repeat and -1 for the second half. Noise picks a new random value every time a repeat begins.
- The frequency changes evenly from the start frequency to the end frequency.
- The volume rises from 0 during the first 4 milliseconds (this avoids a click at the start) and then falls evenly to 0 at the end, which makes the sound fade out like a plucked string.

`note("A4")` turns a note name into a frequency. Going up one note multiplies the frequency by the 12th root of 2, so going up 12 notes (one octave) doubles it. The formula is `440 * 2^(notesAboveA4 / 12)`, because A4 is 440 Hz. `join`, `mix` and `addInto` put arrays of numbers together. Music is made by starting with silence and adding each note at its time. `Sound.fromSamples` turns the array into 16-bit numbers that a `Clip` can play.

## 12. Files, tools and the build

### Where files are loaded from

- Game files are loaded with paths like `"assets/hero.png"`. These paths are relative to the folder the game was started in. `Build.java` starts each game in its own folder, and the start scripts of a packaged game do the same.
- The engine has one file of its own, `font.png`. It sits next to the engine's compiled classes and is loaded with `getResourceAsStream`, which finds files on the class path. This works both from the `out/` folder and from inside a `.jar`.

### Pictures and sounds made by code

`tools/MakeAssets.java` draws the font, the sprites, the tiles and the platformer levels. It always produces exactly the same files, because its random numbers start from a fixed value (a *seed*). `tools/MakeSounds.java` makes the platformer's sounds and music. Its noise sounds use real random

numbers, so they come out slightly different each time. The PNG and WAV files the tools make are saved in the project, so nobody needs to run the tools. They are there so you can see how the files were made, and change them.

### Build.java

`Build.java` is a single Java file, started with `java Build.java`. It:

1. checks that Java 25 or newer is installed, including the compiler;
2. deletes the `out/` folder, compiles `engine/src` into `out/engine` with all compiler warnings switched on, and copies the font next to the classes;
3. compiles each folder in `games/` into `out/games/<name>`. It switches off one warning, "this-escape", which complains about calling methods such as `setName(...)` in a constructor. That is normal in beginner code.
4. If you give it a game name (`java Build.java demo`), it starts that game.
5. If you write `java Build.java package demo`, it builds a `.jar` file containing the engine and the game, copies the game's `assets/` folder, adds start scripts and a README, and zips it all up.

## 13. Error messages

- When a game makes a mistake, the engine throws a `RuntimeException` with a sentence explaining it: which file, what was expected, and where it looked. For example: `Could not find the image 'assets/hero.png'. Looked for it here: /Users/...`.
- Some Java methods can throw *checked* exceptions, such as `IOException`, which normally force you to write `try` and `catch`. The engine catches these itself and turns them into messages like the one above, so game code never needs `try`.
- Methods check their arguments when a wrong value would otherwise cause a confusing error later, for example a negative size or a tile number that does not exist.
- If anything crashes, the window closes properly and the program ends with exit code 1.

## 14. Design decisions

| Decision | Other options | Why this one |
|---|---|---|
| Draw everything ourselves into an `int[]` | let Java2D draw; use OpenGL through a library (LWJGL); use JavaFX | All drawing code can be read and understood, and nothing extra has to be installed. Java2D was ruled out because it hides too much of how drawing works. |
| Use AWT only for the window and input | Swing, JavaFX | AWT is part of every JDK, and Swing is built on top of it anyway. PGE uses none of its buttons or other widgets. |
| 640 x 360 as the default size | 320 x 180, 1280 x 720 | It scales up exactly to the common screen sizes (2x for 720p, 3x for 1080p, 4x for 1440p, 6x for 4K), and it is small enough for our own drawing code to be fast. |

| Decision | Other options | Why this one |
|---|---|---|
| Always 60 updates per second | measure the time since the last update | Game code is simpler, and the game behaves the same on every computer. Pico-8 works this way too. |
| `Input` methods are `static` | pass an input object to the game | `Input.isDown(Key.LEFT)` is easy to read and works anywhere. There is only one keyboard, so one `Input` is enough. |
| Private fields with getters and setters | public fields | This is what the course teaches. Methods like `move(dx, dy)` keep game code short. |
| Positions are `double` | `int` | Speeds like 1.5 pixels per update and gravity need fractions. Positions are rounded down when drawing. |
| Games stop characters at walls themselves | a "move and stop at walls" method in the engine | It is the most interesting part of a platformer to write. The engine provides the questions ( `overlapsSolid` ) but not the answer. |
| Find objects by name and cast them | find them by class, using generics | Names and casts are easy to understand in a first course. |
| Ask buttons `wasClicked()` | have buttons call a method when clicked | No lambdas or anonymous classes are needed, and it works the same way as keyboard input. |
| A build script written in Java | Maven, Gradle, shell scripts | Nothing extra to install, it works on Windows, macOS and Linux, and it can be read like any other Java code. |
| `Clip` for all sounds | stream sounds from the file while playing | Clips are simple to start, stop, repeat and make louder or quieter. They are fine for sound effects and short music. Streaming is left as an exercise. |
| WAV files only | also MP3 and OGG | Java cannot read MP3 or OGG without extra libraries. |
| A synthesizer in the engine | only sounds loaded from files | Games can have sound without any files, and it shows that sound, like pictures, is just numbers. |

## 15. Where to add new features

| Feature | Where to start |
|---|---|
| new drawing methods (triangles, bigger or rotated pictures, see-through colours) | `gfx/Canvas.java` . Follow the same three steps as the other methods: camera, clipping, writing pixels. |
| keeping see-through colours from PNG files | `gfx/Bitmap.java` (the method `fromImage` ) and the picture loops in `Canvas` |
| more letters in fonts | `gfx/Font.java` (the method `toIndex` and the image layout) and `tools/MakeAssets.java` |
| animated tiles, more information per tile | `world/TileMap.java` (the method `draw` and the `solid` array) |
| camera shake, zoom | `world/Camera.java` ( `getX` , `getY` and `applyTo` ) |
| particles, bullets, moving platforms | new subclasses of `GameObject` . Try them in a game first. |

| Feature | Where to start |
|---|---|
| new screen elements (sliders, text boxes) | extend `ui/UIElement`, and read `Input` in `update()` |
| game controllers | Java cannot read game controllers without an extra library |
| sound effects such as fades, left/right speakers or pitch | `audio/Sound.java` and `audio/Synth.java` |
| long music played straight from the file | a new class built around `SourceDataLine`, with its own thread |

When you change the engine, keep to its style: easy-to-read code, comments that explain why, and error messages in plain English. Make sure `java Build.java` shows no warnings. Section 11 of the Guide has exercises with hints.

## 16. What the engine cannot do yet

- Pictures cannot be partly see-through, rotated or made bigger (only text can be made bigger).
- The default font only has English letters. Other letters are shown as a box.
- Game pictures much bigger than 960 x 540 become slow.
- Sound comes equally out of both speakers, only WAV files can be loaded, and the whole sound is kept in memory.
- Only one mouse or touch point is followed. There is no support for multi-touch or game controllers.
- Levels can only be loaded from CSV files, not from Tiled's own `.tmx` files.
- There is no built-in physics, collision handling or path finding. This is on purpose (see section 14).